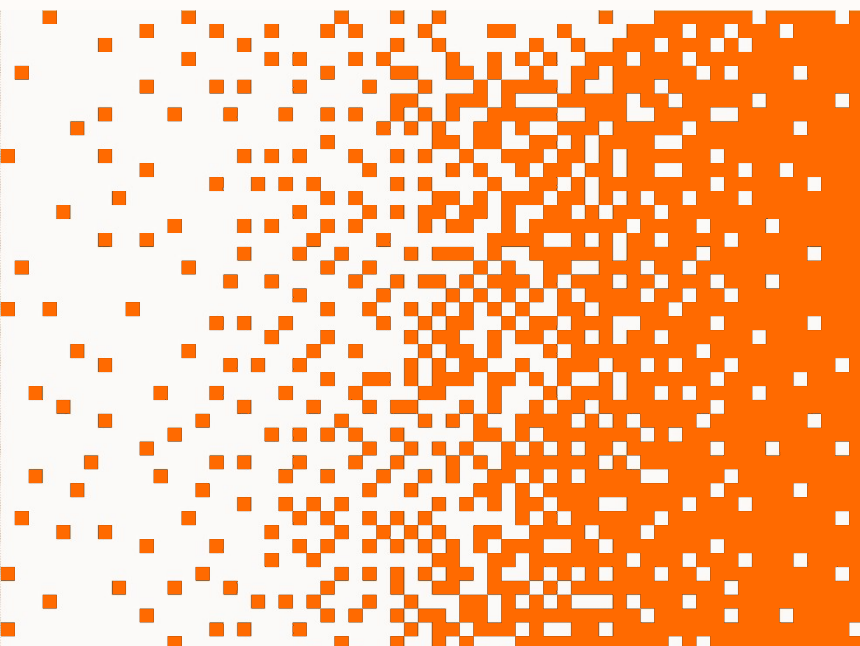


Von der Miniwelt zur laufenden Datenbank — eine Kontaktverwaltung mit SQLite

Fallstudie mini-contacts



Kontaktverwaltung: der ganze Datenbankentwurf im Kleinen

Sechs Schritte führen von der Miniwelt zur laufenden Anwendung



Datenbankentwurf ist ein Weg mit festen Etappen: Erst wird die Miniwelt beschrieben, dann entsteht das ER-Modell, die Normalisierung entfernt Redundanz, das relationale Modell übersetzt in Tabellen, SQLite setzt sie physisch um — erst am Ende steht die Anwendung. Wer die Reihenfolge einhält, baut Strukturen, die Änderungen überleben.



Jede Etappe hat ein Ergebnis, das die nächste braucht — Abkürzungen rächen sich spätestens beim ersten Schemaumbau.

Die Miniwelt legt fest, was die Datenbank abbilden muss



Die Miniwelt ist der Ausschnitt der Realität, den die Datenbank tragen soll — hier reichen vier Sätze, um sie zu beschreiben. Ihre Substantive werden später Entitäten, ihre Verben Beziehungen, und Mengenangaben wie "beliebig viele" legen die Kardinalitäten fest. Wer die Miniwelt sauber aufschreibt, hat den halben Entwurf.

Kontakte

Vorname, Nachname,
optional Geburtstag und
Notiz — die zentrale Entität.

Kanäle

Mobil, Festnetz, E-Mail, Web
— beliebig viele je Kontakt.

Adressen

Privat, Arbeit, sonstige —
keine, eine oder mehrere je
Kontakt.

Gruppen

Familie, Arbeit, Verein — ein
Kontakt in mehreren
Gruppen, eine Gruppe mit
vielen Kontakten.

Miniwelt in vier Sätzen: Substantive werden Entitäten, Verben werden Beziehungen, Mengenangaben werden Kardinalitäten.

Das ER-Modell beschreibt die Miniwelt vor jeder Tabelle

Chen-Notation trennt Entitäten, Beziehungen und Attribute



Die Chen-Notation kennt genau drei Bausteine, und jeder hat eine feste Form: Rechtecke für Entitätstypen, Rauten für Beziehungstypen, Ellipsen für Attribute — Schlüsselattribute werden unterstrichen. Diese Trennung zwingt dazu, vor der ersten Tabelle zu klären, was ein Ding, was eine Verbindung und was nur eine Eigenschaft ist.



Rechteck — Entitätstyp

Ein Ding der Miniwelt mit eigener Identität: Kontakt, Gruppe, Adresse.



Raute — Beziehungstyp

Verbindet Entitätstypen; die Kardinalität steht an den Kanten: 1:1, 1:N oder N:M.

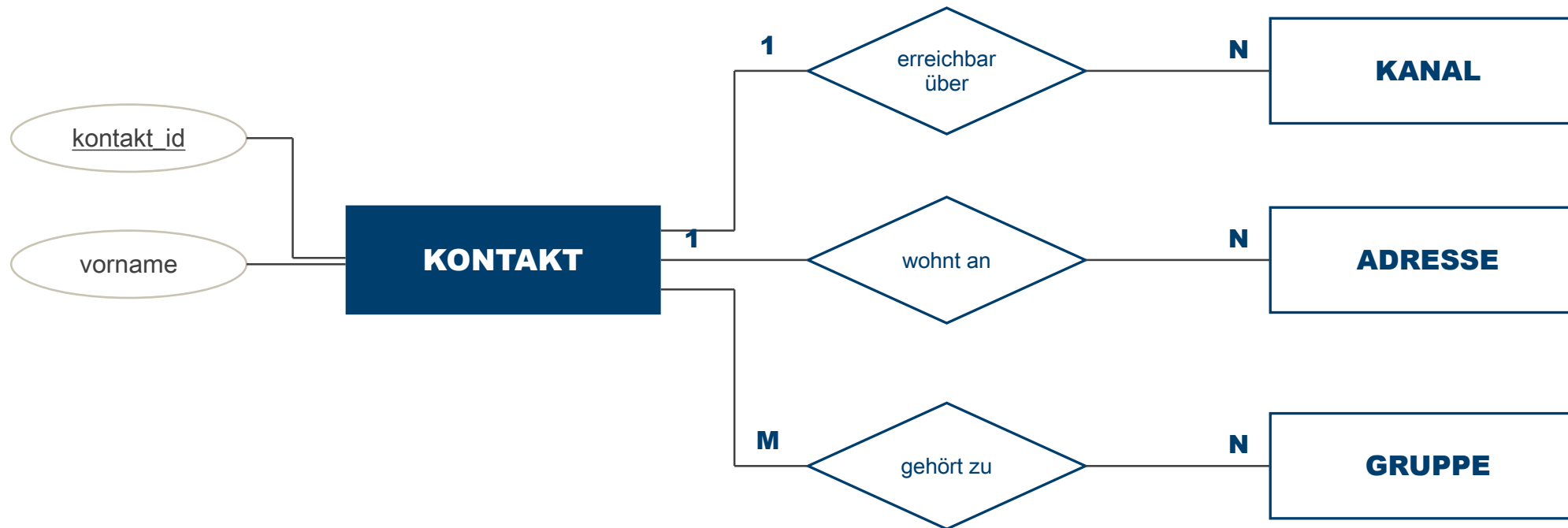


Ellipse — Attribut

Eigenschaft einer Entität; das unterstrichene Schlüsselattribut identifiziert eindeutig.

Erst die Miniwelt in diese drei Bausteine zerlegen — Tabellen sind Schritt vier des Entwurfswegs, nicht Schritt eins.

Ein Kontakt hat viele Kanäle und Adressen und gehört zu vielen Gruppen



Schlüsselattribute unterstrichen; die übrigen Attribute je Entität stehen im Repo (docs/02_er_modell.md).

Eigene Darstellung

Normalisierung entfernt Redundanz Schritt für Schritt

Die flache Kontaktliste erzwingt Redundanz und Anomalien



Als einzelne Tabelle gedacht, mischt die Kontaktliste mehrere Werte in eine Zelle und wiederholt denselben Namen Zeile für Zeile. Solche Strukturen erzeugen drei typische Anomalien: Ändern trifft nie alle Kopien, Löschen nimmt Wissen mit, und Einfügen braucht Zeilen, die es noch gar nicht gibt.

name	telefon	adresse	gruppen
Anna Müller	+49 171 111, +49 931 222	Hauptstr. 5, 97070 Würzburg	Familie, Verein
Ben Bauer	+49 172 333	Mainkai 53, 97082 Würzburg	Arbeit
Anna Müller	+49 171 111	Gartenweg 2, 97318 Kitzingen	Familie

Änderungsanomalie

Annas Nummer steht in mehreren Zeilen — eine vergessene Kopie macht den Bestand widersprüchlich.

Löschanomalie

Mit Bens letzter Zeile verschwindet auch das Wissen, dass es die Gruppe "Arbeit" gibt.

Einfügeanomalie

Eine neue Gruppe ohne Mitglied lässt sich gar nicht erfassen — es gibt keine Zeile dafür.

Die erste Normalform macht jeden Attributwert atomar



Die erste Normalform verlangt genau einen Wert je Zelle. Listen wie "Familie, Verein" werden zu einzelnen Zeilen, zusammengesetzte Angaben wie die Adresse in ihre Bestandteile zerlegt. Die Daten werden dadurch erst durchsuchbar und sortierbar — die Redundanz aber wächst zunächst sogar noch an.

✗ Vorher: Listen in der Zelle

telefon = "+49 171 111, +49 931 222"

gruppen = "Familie, Verein"

Eine Zeile je Kontakt — nicht durchsuchbar, nicht sortierbar, nicht einzeln löscher.

✓ Nachher: eine Zeile je Wert

kanal(1, mobil, "+49 171 111")

kanal(1, festnetz, "+49 931 222")

Jeder Wert atomar in einer eigenen Zeile — Struktur sauber, Redundanz beseitigen erst 2NF und 3NF.

Merksatz: 1NF ordnet die Struktur — gegen Redundanz helfen erst die nächsten Normalformen.

2NF und 3NF lösen versteckte Abhängigkeiten auf



Nach der 1NF hängen noch Attribute am falschen Ort: Der Name hängt nur an der Kontaktnummer statt am ganzen zusammengesetzten Schlüssel, und der Ort hängt über die Postleitzahl nur mittelbar daran. 2NF und 3NF zerlegen die Tabelle entlang dieser Abhängigkeiten in eigene Relationen.

2NF — ganzer Schlüssel

➔ Name und Geburtstag hängen nur an kontakt_id — sie wandern in die eigene Tabelle kontakt.

3NF — nichts Transitives

➔ plz → ort: Der Ort hängt am Schlüssel nur über die Postleitzahl — streng genommen gehört er in eine Ort-Tabelle.

Pragmatik — bewusst abweichen

➔ Wir behalten plz und ort zusammen: dokumentierte Entscheidung im Schema statt blinder Regeltreue.

Normalisierung ist ein Werkzeug, um Redundanz zu erkennen — nicht jede erkannte Redundanz muss beseitigt werden.

Vom ER-Modell zum relationalen Modell

Drei Regeln übersetzen das ER-Modell in Tabellen



Die Übersetzung ins relationale Modell folgt festen Regeln und lässt keinen Spielraum: Entitätstypen werden Tabellen, 1:N-Beziehungen werden zum Fremdschlüssel auf der N-Seite, und N:M-Beziehungen brauchen eine eigene Koppeltabelle, weil sie sich in keiner einzelnen Spalte unterbringen lassen.

Entität → Tabelle

KONTAKT wird zur Tabelle kontakt;
das Schlüsselattribut wird
Primärschlüssel kontakt_id.

1:N → Fremdschlüssel

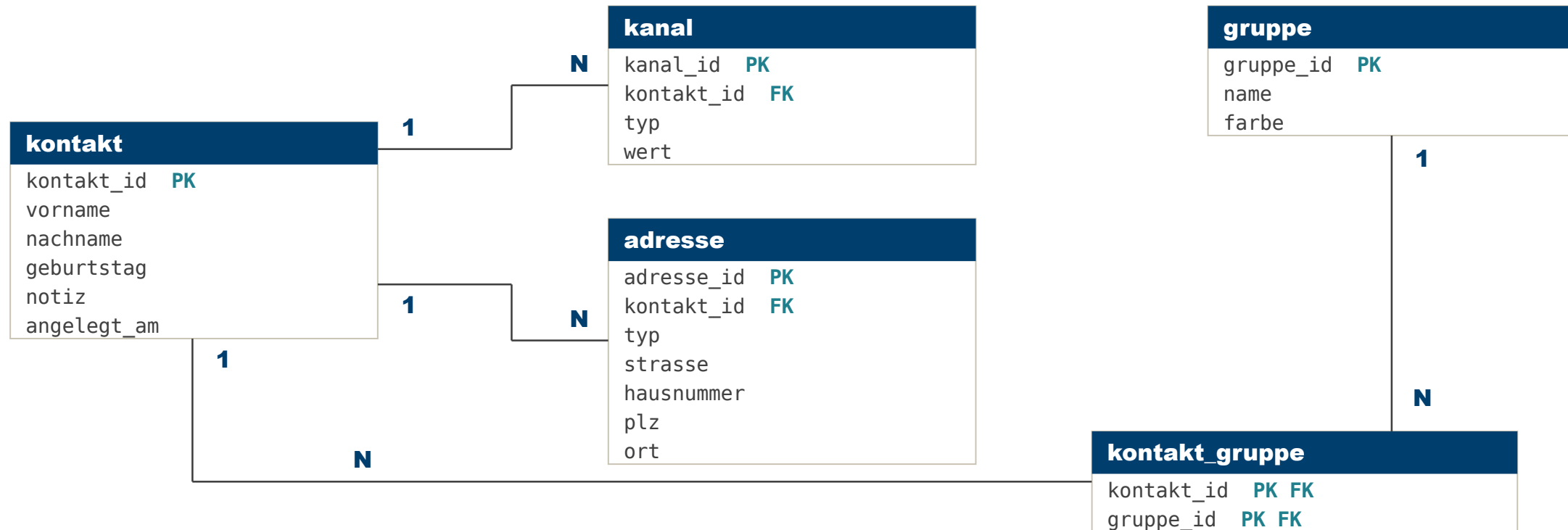
kanal.kontakt_id verweist auf kontakt —
die N-Seite trägt den Schlüssel der 1-
Seite.

N:M → Koppeltabelle

kontakt_gruppe(kontakt_id, gruppe_id):
zwei Fremdschlüssel, zusammen der
Primärschlüssel.

Merkregel: Beziehungen verschwinden nicht — sie werden zu Schlüsseln.

Fünf Tabellen tragen das gesamte Datenmodell



Die N:M-Beziehung aus dem ER-Modell lebt jetzt in der Koppeltabelle — zu beiden Seiten je eine 1:N-Beziehung.

Eigene Darstellung

SQLite: die Datenbank ohne Server

SQLite ist eine Datei im Prozess der Anwendung — kein Server



Klassische Datenbanksysteme laufen als eigener Serverprozess mit Netzwerk, Konten und Administration. SQLite verfolgt das Gegenmodell: Die komplette Datenbank ist eine einzige Datei, die Bibliothek läuft im Prozess der Anwendung — SQL, Schlüssel und Normalformen funktionieren dabei identisch zu den großen Systemen.

▶ Client-Server (PostgreSQL, MySQL)

eigener Serverprozess, Zugriff über das Netzwerk
Benutzerverwaltung und Rechte
viele gleichzeitige Schreiber — zentrale Unternehmensdaten

▶ SQLite (eingebettet)

eine Datei: contacts.db — kopieren ist Backup
in Python eingebaut: `import sqlite3`
ein Schreiber zur Zeit — lokale Apps, Mobile, Browser

SQLite steckt in jedem Smartphone und Browser — die am weitesten verbreitete Datenbank der Welt. Für die Lehre heißt das: gleiches SQL, null Aufbau.

CREATE TABLE setzt das Modell wörtlich in SQL um



Das physische Schema ist die wörtliche Übersetzung des relationalen Modells: je Tabelle ein CREATE TABLE, je Beziehung ein Fremdschlüssel. Regeln, die immer gelten sollen, gehören in das Schema — Wertebereiche als CHECK, Eindeutigkeit als UNIQUE, Löschverhalten als ON DELETE CASCADE.

```
CREATE TABLE kanal (  
    kanal_id    INTEGER PRIMARY KEY,  
    kontakt_id  INTEGER NOT NULL  
                REFERENCES kontakt(kontakt_id)  
                ON DELETE CASCADE,  
    typ         TEXT NOT NULL  
                CHECK (typ IN ('mobil', 'festnetz',  
                               'email', 'web')),  
    wert        TEXT NOT NULL  
);
```

1 ON DELETE CASCADE: Kanäle sterben mit ihrem Kontakt — die Datenbank räumt auf, nicht die App.

2 CHECK erzwingt den Wertebereich direkt im Schema — ein Enum auf Datenbankebene.

Ohne PRAGMA prüft SQLite keine Fremdschlüssel



Aus Kompatibilität zu alten Versionen ist die Fremdschlüsselprüfung in SQLite standardmäßig abgeschaltet — REFERENCES ist dann nur Dokumentation. Eingeschaltet wird die Prüfung je Verbindung; der Aufruf gehört deshalb an genau eine Stelle: direkt neben das Öffnen der Datenbank.

✗ Pragma vergessen: stille Fehler

INSERT mit kontakt_id 9999 gelingt, DELETE hinterlässt verwaiste Kanäle — kein Fehler, aber falsche Daten.

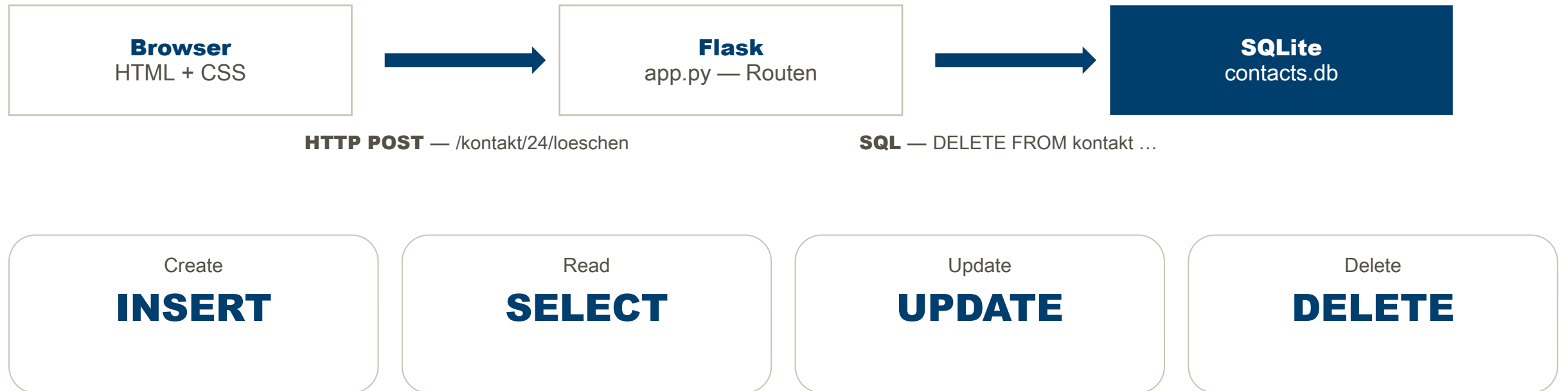
✓ Pragma gesetzt: laute Fehler

"FOREIGN KEY constraint failed" — der Fehler kommt sofort und zeigt auf die Ursache statt auf Symptome.

```
con = sqlite3.connect("contacts.db")
con.execute("PRAGMA foreign_keys = ON")    # bei jedem Connect
```

Ei ne kl ei ne Web-App macht CRUD gr ei f bar

Jeder Klick in der Oberfläche wird zu genau einem SQL-Befehl

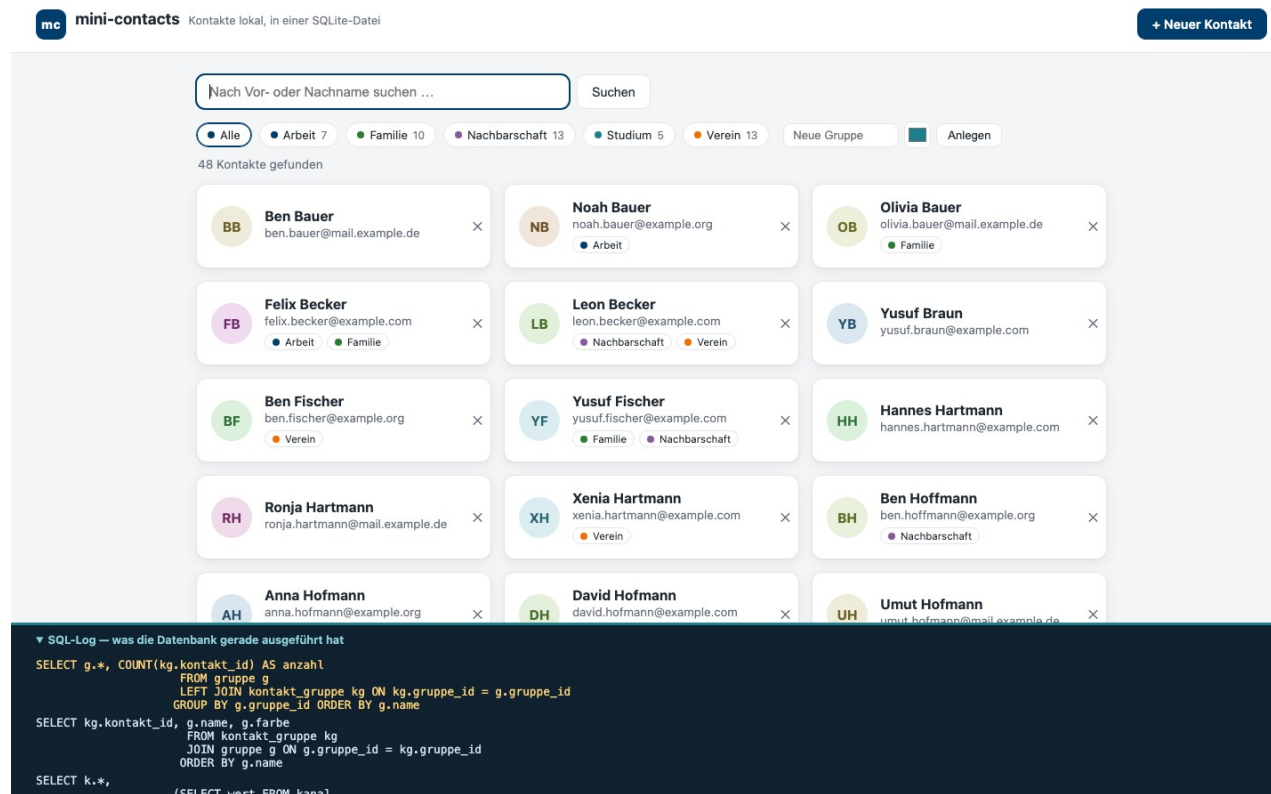


Vier Grundoperationen — mehr braucht eine datengetriebene Anwendung nicht, und jede hat ihr SQL-Gegenstück.

Die App zeigt zu jedem Klick das ausgeführte SQL



Das didaktische Herzstück der App ist das SQL-Log am unteren Rand: Es zeigt die zuletzt ausgeführten Statements mit eingesetzten Parametern. Suche, Gruppenfilter und jedes Speichern lassen sich so live auf ihr SQL zurückführen — die Oberfläche wird zur Übungsumgebung für die Vorlesung.

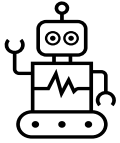


1 Gruppen-Chips filtern die Liste — dahinter liegt ein JOIN über die Koppeltabelle kontakt_gruppe.

2 SQL-Log am unteren Rand: die letzten Statements, das jüngste zuoberst.

Eigene Darstellung (mini-contacts, Stand 08/2026)

Fake-Daten kommen aus Generatoren — nie aus echten Adressbüchern



Eine leere Datenbank lässt sich weder testen noch vorführen, echte Personendaten sind tabu — Testdaten werden deshalb generiert. Drei Wege führen zum Ziel: konfigurierbare Generatoren im Browser, Programmbibliotheken im eigenen Skript und KI-Assistenten, die das Schema samt Constraints verstehen.

Mockaroo (Browser)

Spalten und Datentypen zusammenklicken, bis 1 000 Zeilen gratis — Export als CSV, JSON oder direkt als SQL-INSERTs.

Faker (Python)

`pip install faker`, `Locale de_DE` — Namen, Adressen und Mails in beliebiger Menge, reproduzierbar per Seed.

KI-Assistenten

Schema hingeben, INSERTs generieren lassen — versteht CHECK und Fremdschlüssel; Ergebnis immer gegen die Datenbank laufen lassen.

Datenschutz: Fake-Daten sind Pflicht, sobald ein Repo öffentlich wird — E-Mail-Domains nach RFC 2606 (example.org).

Das Repo `mini-contacts` enthält den ganzen Weg zum Nachbauen

Struktur

```
mini-contacts/
├── app.py           # Flask-App + SQL-Log
├── seed.py          # Fake-Daten-Generator
├── db/schema.sql    # das physische Schema
├── docs/            # Entwurfsweg Schritt 1-7
│                   #   mit Mermaid-Diagrammen
├── slides/          # dieses Foliendeck
├── static/          # Styling
└── templates/       # HTML (Jinja2)
```

Schnellstart (Win / macOS / Linux)

```
git clone github.com/swrobuts/
    mini-contacts.git
cd mini-contacts
pip install flask
python seed.py      # 48 Demo-Kontakte
python app.py
# -> http://127.0.0.1:5050
```

github.com/swrobuts/mini-contacts — Fragen und Fundstücke gern als Issue.